Product Requirements Document - ji-analysis

Author: Ramnish **Date:** 2026-05-05

Document Map

This PRD describes NJI (New JI) — a greenfield rebuild of HMCTS's Judicial Itineraries system on a 12-service API-driven architecture with a modern UI, deployed on Azure, replacing the unsupported Oracle APEX (OPT) platform. The build is in isolation from APEX; cutover is phased per region.

The document is organised so each section answers a distinct question:

Section	Question it answers
Executive Summary	What is NJI, and why is it being built?
Project Classification	What kind of project is this (type / domain / complexity)?
Success Criteria	What does "successful" look like, and how is it measured?
Product Scope	What's in MVP, what's growth, what's vision?
User Journeys	How do real users flow through NJI to do their work?
Domain-Specific Requirements	What UK govtech compliance / technical / integration constraints apply?
API Backend Specific Requirements	What does the 12-service API surface look like?
Project Scoping & Phased Development	How do MVP philosophy, build phases, and rollout waves fit together?
Functional Requirements (FR1–FR61)	The binding capability contract.
Non-Functional Requirements (NFR1–NFR42)	The binding quality-attribute contract.
Decisions Log (D1–D9)	The 9 programme-level decisions that govern scope and approach.
Glossary, References	Acronyms and source documents.

Executive Summary

JI (Judicial Itineraries) is HMCTS's system for planning, allocating, confirming, and paying for judicial sittings across Civil, Family, and Crown Courts. It is owned today by an unsupported Oracle APEX (OPT) platform and Board-endorsed for full replacement.

This PRD describes the **greenfield rebuild** of JI — referred to throughout as **NJI (New JI)** — as a modern API-driven application. NJI replicates the functional surface of the existing APEX system on a 12-service decomposition (Domain / Cross-cutting / Read-model clusters), with a modern UI replacing the legacy APEX UI, and exposes first-class APIs that the wider HMCTS ecosystem (DA&I, finance, future Tribunals

tooling, Actuals programme, Scheduling & Listing reforms) consumes directly — replacing today's brittle export-file-by-email integration model.

Target users (~11 roles, scoped by Region and Area):

- RSU / Judicial Team (Admin, Full Access, Verifier / Read-only)
- Court users (Full Access, Enhanced CJ, Limited / Read-only)
- Judges, Judges' Clerks, Presiding Judges / Clerks
- Finance / Payment Authoriser
- MI / Reporting User

Operational platform support (formerly OPT Support in the legacy system) is handled outside JI by external HMCTS roles and is not modelled as a JI user role.

The problem being solved:

1. OPT / APEX is unsupported legacy with a fixed end-of-life. Continuing investment is investment in a platform that cannot be operated long-term.
2. The current export-only integration model (Excel, PDF, email) cannot scale to support pending HMCTS programmes (Tribunals coverage, Actuals, Scheduling & Listing reforms). Each new consumer must wedge itself into a manual export workflow.
3. The legacy UI, while functional, is APEX-era — the user-experience baseline is set by 2000s-vintage tooling rather than a modern, accessible, performant interface.

What success looks like: every region migrated to NJI; APEX retired; downstream consumers integrating via API rather than manual export; future HMCTS programmes building on JI's APIs as a strategic platform.

What Makes This Special

This is a deliberately simplified greenfield rebuild, not a strangler decomposition or a cosmetic refresh. Five characteristics distinguish it:

1. **Greenfield-in-isolation, not strangler.** Oracle APEX is not amenable to strangler decomposition. NJI is built end-to-end before any user moves; APEX continues unchanged for non-migrated regions during phased rollout. This single decision cascaded into every architectural simplification — no dual-write, no event bus, no synchronisation layer.

2. **Aggressive simplification with explicit roadmap pay-back.** REST-first synchronous coordination, Strategy A federated read models (Itinerary, MI Feed), no domain event stream, no webhook surface, log-based audit and observability for MVP (D7) with structured user-action audit on the post-MVP roadmap. Each simplification is a deliberate trade — buying delivery velocity without pretending the trade isn't there.

3. **APEX as behavioural reference for acceptance testing (D5).** Real APEX runs as the oracle; NJI is the system under test. Acceptance tests verify behavioural parity — not just functional parity — capturing edge cases that the documentation does not.

4. **Phase 0 as platform smoke-test.** Reference Data and Users + Roles are migrated from APEX in Phase 0 (D3 + D9). API-as-Product standards (versioning, `/capabilities`, deprecation policy) are

battle-tested on Reference Data writes and Authorisation lookups before any domain service is built. Cross-cutting concerns are exercised before the team commits to the domain build pattern.

5. **Per-region, full-role phased cutover (D8).** Each rollout wave is a discrete event — a whole region with all applicable user roles moving together. Risk is amortised across waves rather than concentrated at a single big-bang. Migrated users do not use APEX; non-migrated users do not use NJI. No contention, no synchronisation.

Why now: OPT is unsupported; HMCTS programmes (Tribunals, Actuals, Scheduling & Listing) require integration patterns the export-only legacy cannot support; every month JI stays on APEX is a month the wider ecosystem cannot integrate via API.

Project Classification

Dimension	Value
Project Type	`api_backend` — the 12-service API decomposition is the durable product surface
Project Type override	`ux_ui`, `visual_design`, `user_journeys` remain in scope per D4 (modern UI replicates APEX layouts using a modern UI stack)
Domain	`govtech` (UK HMCTS — judicial operations)
Domain notes	UK-translated compliance: HMCTS / WCAG accessibility, GDS service standard, UK GDPR, FOI / transparency. CSV's US-specific concerns (FedRAMP, Section 508) do not apply.
Complexity	`high`
Project Context	`brownfield-rebuild`
Classification rationale	Comparative Analysis Matrix from Step 2 Advanced Elicitation: `api_backend` 24, `web_app` 22, `saas_b2b` 18; `govtech` 29, `legaltech` 10. The `api_backend` + UX-override is the cleanest workaround for the CSV's structural mismatch with API-first products that have first-class UI scope.

Success Criteria

User Success

For each role on NJI, the equivalent legacy workflow can be completed without re-training, and faster or no slower than APEX. Specifically:

- **RSU / Judicial Team**: maintain judge profiles, working patterns, tickets, absences, and vacancies via modern UI; vacancy auto-creation from approved absences works end-to-end (R4 from architecture); no manual reconciliation step needed beyond the legacy baseline.
- **Court users**: confirm sittings and bookings (the daily operational task) with comparable or fewer clicks than APEX; AM/PM split, work-type editing, and verifier sign-off (County Courts) preserved.
- **Judges and Judges' Clerks**: view itinerary and forward look filtered to authorised judges only (R2 — Authorisation gates every read); no case-level data exposure.

- **Finance / Payment Authoriser**: receive JFEPS-compatible Excel payment schedule via the same email mechanism as APEX (zero-change-for-finance); JFEPS schedule shape unchanged.
- **MI / Reporting**: standard reports run with same parameter filters as APEX; copy-to-Excel and PDF export preserved; aggregated-only, no case-level detail (REP-BR-NFR-03 from `functional-modules.md`).
- **All roles**: modern UI on new technology delivers a measurable usability improvement over APEX-era UI — accessibility (HMCTS / WCAG), responsiveness, performance baseline at least matches APEX page-level NFRs (≤ 5 s dashboard refresh, ≤ 10 s list/filter, ≤ 15 s batch/annual, ≤ 30 s reports/Forward Look).

Business Success

- **APEX retirement** — every region migrated to NJI; Oracle APEX (OPT) decommissioned. Programme is not "successful" until the last region cuts over.
- **Strategic integration platform** — at least one external HMCTS programme (Tribunals coverage, Actuals, or Scheduling & Listing) integrating with JI via API by `TBD post-MVP date`, replacing manual export workflow.
- **Continuity of operations** — zero unpaid judges due to migration. Payment exports to JFEPS / Liberata continue uninterrupted across every rollout wave.
- **Programme delivery** — phase-by-phase delivery cadence (Phase 0 → 8 build, then per-region rollout) on the schedule the programme commits to. Specific dates are programme-management territory and not set in this PRD.

Technical Success

- **All 12 services live** — Reference Data, Authorisation (with SSO), Configuration, Notification, Judge, Absence, Vacancy, Booking, Sitting, Payment, Itinerary, MI Feed deployed and operating per Phase 0 → 8.
- **Phase 0 migration correctness** — 100% of in-scope APEX Reference Data lists migrated and signed off by RSU / judicial-team owners (D3, Risk #13). 100% of active APEX user records migrated and successfully mapped to IdP principals (D9, Risk #14); records that don't reconcile have an explicit handling decision (drop / hold / manual map), zero migrated as ambiguous.
- **Behavioural parity with APEX** (D5) — every domain service has acceptance tests where real APEX runs as the oracle and NJI is the system under test; behavioural-parity tests pass before each rollout wave.
- **API-as-Product standards** in force from Phase 0 — every domain and read-model service exposes a versioned contract, `/capabilities`, and a documented deprecation policy.
- **Performance NFRs match or exceed APEX**: ≤ 5 s dashboard refresh; ≤ 10 s list / filter; ≤ 15 s batch / annual; ≤ 30 s reports and Forward Look (HOME-NFR, MJ-NFR, ABS-NFR, VAC-NFR, FPB-NFR, PAY-NFR, SIT-NFR, REP-BR-NFR, JFL-NFR from `functional-modules.md`).
- **Strategy A federated read models** (Itinerary, MI Feed) meet their NFRs without requiring Strategy C cache fallback in the MVP — or the cache fallback is implemented and switched on by the time of the wave that needs it (Risk #9).
- **Log-based audit / observability minimum** (D7) operational from Phase 1 onwards — structured logging, correlation IDs, error categorisation, retention sufficient for pilot incident triage.

Measurable Outcomes

Outcome	Target	Source
Reference Data migration accuracy	100% of in-scope lists, signed off by named owners	D3 + Risk #13
User-record migration accuracy	100% of active APEX users mapped to IdP principal; zero ambiguous records	D9 + Risk #14
Payment export continuity at cutover	Zero failed JFEPS payment cycles attributable to migration	Business success criterion above
Behavioural parity per domain service	100% of APEX-as-oracle acceptance tests pass before that wave's cutover	D5
Per-wave feature parity	100% of in-region role workflows demoed and signed off before wave cutover	D8 + Risk #3
Page-level performance	All page-level NFRs from `functional-modules.md` met or exceeded	functional-modules.md cross-cutting NFRs
Forward Look (federated read)	≤ 30 s for a Region under Strategy A; Strategy C cache fallback designed	JFL-NFR-01, Risk #9
API consumer onboarding (post-MVP)	At least one external HMCTS programme integrating via API by `TBD`	Vision: strategic integration platform
MVP user-action audit	Not in MVP; on roadmap (D7)	D7

Product Scope

MVP — Minimum Viable Product

The MVP is the smallest deliverable that supports phased per-region rollout (D8). It comprises:

- **Phase 0 Foundations** (D1, D7, D9): Reference Data + Users/Roles migrated from APEX; Authorisation with SSO; Configuration; Notification; API contracts (versioned + paper contracts for Itinerary / MI Feed); deployment platform (CI/CD); structured logging conventions (D7); stub Home / navigation shell.
- **All 12 services built** (Phases 1–8): Judge, Absence, Vacancy, Booking, Sitting, Payment (incl. Reconciliation), Itinerary, MI Feed.
- **Modern UI for all 11 user roles** replicating APEX layouts (D4) — every domain phase delivers its corresponding APEX module(s) end-to-end.
- **Phase 9 — Pilot rollout (wave 1)**: one region migrates, all applicable roles, with feature-parity gating per Risk #3.
- **Behavioural parity with APEX** verified through acceptance tests using APEX as oracle (D5) for every domain service.
- **Log-based audit and observability** (D7) — application logs only; no metrics platform, no traces, no structured user-action audit.

Explicit exclusions from MVP (post-MVP roadmap):

- Structured user-action audit (D7 roadmap)
- Metrics + traces + dashboards observability (D7 — log-based MVP only)
- Domain event stream / webhooks (architecturally rejected for MVP — REST-first)
- Active matching / allocation service (architecturally deferred)
- Bi-temporal history
- Tribunals coverage
- Historical-data access for migrated users (D3 + Risk #2 — separate decision)

Growth Features (Post-MVP)

- **Wave-by-wave rollout**: Phase 10..N — additional regions migrate, wave by wave, until all regions are on NJI and APEX is retired.
- **Structured user-action auditing** (D7 roadmap commitment) — who did what, when, with before/after values for write operations.
- **Full observability** — metrics + traces + dashboards beyond the log-based MVP minimum.
- **External API consumer onboarding** — DA&I migrates from export-based MI to API-based MI Feed; future programmes (Tribunals, Actuals, Scheduling & Listing) onboard onto JI's APIs.
- **Historical-data access** policy for migrated users — read-only APEX bridge, or one-shot export, or policy-of-no-access (Risk #2).
- **Cross-region workflow handling** matures from per-wave manual coordination (Risk #1) to system-supported flows.

Vision (Future)

- **Strategic integration platform** for HMCTS judicial scheduling — Tribunals, Magistrates, Civil, Family, Crown all served by a single API-driven foundation.
- **Real-time data flow** to downstream systems (DA&I, finance, performance teams) — potentially via event streams or webhooks if integration patterns demand it (architecturally deferred today, revisitable when justified).
- **Active matching / allocation** service for fee-paid judges to vacancies, beyond today's filter-as-hint approach.
- **Automated reconciliation feed** from JFEPS to JI, replacing today's manual flag-as-reconciled step.
- **Bi-temporal / audit-grade compliance trail** if regulatory scope demands it (today out of scope; revisitable).

User Journeys

Journey 1 — RSU Admin: cover-creation through payment (the canonical operational cycle)

Persona: Sam, an RSU Admin in a regional office. Sam runs the daily operational rhythm — converting absences into vacancies, allocating fee-paid judges, confirming bookings, generating payment schedules. Today Sam does this in APEX, juggling tabs, copying between screens, and sending payment files via email.

Opening scene. A Court office logs an absence request for a salaried judge — leave with cover required. The request lands in Sam's "Outstanding Actions" tile on the Home dashboard.

Rising action.

1. Sam opens the absence record from the dashboard tile. NJI shows the judge's profile, the requested dates, the work-type and ticket fields the Court captured, and a single *Approve* action.
2. Sam approves. The system auto-creates a vacancy (per R4 — Absence approval triggers Vacancy creation), pre-populated with the judge type, work type, ticket, and dates. The vacancy appears in the Vacancies list with a *Needs allocation* status.
3. Sam advertises the vacancy out-of-system using their own mailing list (advertising is not automated — same as APEX). A fee-paid judge replies confirming availability.
4. Sam opens the vacancy, clicks *Create Booking*, picks the fee-paid judge, fills the booking session details. The system creates the booking and synchronously marks the vacancy as filled (R5 — Booking orchestrates `Vacancy.markFilled`). An acknowledgement email is queued to the booked judge.
5. The Court confirms the sitting after it occurs. The booking moves to *Confirmed* status and becomes eligible for payment.
6. At end-of-week, Sam runs *Process Payments*. The system displays the proposed schedule, Sam picks an authoriser, the system emails the JFEPS-compatible Excel to the authoriser. Same email mechanism as APEX, same JFEPS shape, no change for finance.

Critical moment. Step 2 — the absence-to-vacancy auto-creation. In APEX this is a manual click-through dance across multiple screens; on NJI the workflow is one click and the vacancy lands pre-populated.

Resolution. Sam completes the cycle in fewer clicks than APEX, with the same JFEPS output landing at the same finance team. The new reality: the operational task that defined Sam's morning takes less time and produces the same downstream artifacts.

Capabilities revealed: Authorisation gating per role + Region/Area; Absence service with approval workflow; Vacancy auto-creation triggered by Absence; Booking with `Vacancy.markFilled` orchestration; Payment service with JFEPS-shaped Excel content-type; Notification service for booking acknowledgements; Home dashboard with role-scoped Outstanding Actions.

Journey 2 — Court user: daily sitting confirmation

Persona: Priya, a Court user with Full Access in a Crown Court office. Priya's daily task is to confirm yesterday's sittings — verify that they took place, record the actual work type, adjust session duration if needed. Confirmed sittings drive both payment processing (for fee-paid recorders) and MI reporting.

Opening scene. Priya logs in via SSO. The Home dashboard shows a *Sittings awaiting confirmation* tile scoped to her office.

Rising action.

1. Priya clicks the tile, opens the sittings list filtered to *yesterday, this office*.
2. For each sitting: confirm with one click if the planned work-type and session held; or open the row, change work-type from *Crime* to *Civil* (a real example — "remembered" on confirmation per functional-modules.md line 422), split AM/PM if the day was split.
3. For one fee-paid Recorder, the booking required confirmation rather than a sitting; she confirms via the Bookings list with the same one-click flow.
4. Priya finishes the day's confirmations in under five minutes.

Critical moment. The list view itself — fast, filterable, modern UI — vs. APEX's older grid. Same data, same actions, lower friction.

Resolution. Yesterday's data is locked in for payment and MI before Priya's first coffee.

Capabilities revealed: Sitting confirmation with work-type override and AM/PM split; Booking confirmation as a Court-user action distinct from RSU; Authorisation scoping confirmations to the user's office; Home dashboard with outstanding-confirmation tiles.

Journey 3 — Judge: view itinerary and request absence

Persona: Justice Hawthorne, a salaried Circuit Judge. Justice Hawthorne uses JI to see their planned sittings and request absences (training, leave). Today they do this in APEX with a UI that's functional but visually dated and inconsistent on a tablet.

Opening scene. Justice Hawthorne logs in via SSO on a tablet between hearings. The Judge Itinerary view loads scoped to their own profile (per R2 — Authorisation gates every read; judges see only their own itinerary).

Rising action.

1. The itinerary renders cleanly on the tablet — accessible, responsive, performant. APEX's session timeout warnings and grid-style layout are gone.
2. Justice Hawthorne sees a planned training day next month, realises a clash, opens *Request Absence*.
3. The form is short: dates, type (training), notes. Submit.
4. The system routes the request to RSU for approval (via the Court-raised pattern in functional-modules.md §4.5). Acknowledgement email goes out (Notification service).

Critical moment. The itinerary on a tablet — APEX is desktop-first and the difference is felt immediately by every judge using a mobile device.

Resolution. Absence request lands with RSU; Justice Hawthorne moves to the next hearing without losing time to a clunky UI.

Capabilities revealed: Modern, responsive, accessible UI per D4 — the user-experience uplift; Authorisation scoping to *own profile only*; Absence request routing to RSU approval queue; Notification service for acknowledgements.

Journey 4 — DA&I analyst: consume MI Feed API instead of Excel exports

Persona: Riya, a DA&I analyst building monthly utilisation dashboards for HMCTS leadership. Today Riya gets sitting and utilisation data from JI by running APEX reports, copy-pasting to Excel, transforming, and feeding her dashboard tooling. The export-by-Excel chain is brittle, slow, and sensitive to APEX UI changes.

Opening scene. Post-MVP, JI exposes the MI Feed API. Riya pulls her API credentials (her IdP principal authorised by JI's Authorisation service) and writes a small script.

Rising action.

1. Riya calls `GET /reporting/sittings` with parameters for region, judge type, date range. The API returns aggregated, case-level-stripped JSON (REP-BR-NFR-03 — no case-level exposure).

2. The same call replaces three previous APEX-export-and-transform steps.
3. Riya schedules the script to run nightly. The dashboard now updates without her copy-paste.
4. When MI Feed's contract evolves, the versioned content-type and `/capabilities` (per API-as-Product standards) tell Riya what changed and when.

Critical moment. The first nightly run that lands without Riya touching it. The export-and-transform manual chain is gone.

Resolution. DA&I's monthly reporting cycle accelerates; future programmes (Tribunals, Actuals) onboard onto the same APIs without inventing new export workflows. The strategic-platform vision becomes operational.

Capabilities revealed: MI Feed (Read-model service, Strategy A pull-based federation); versioned API contracts with `/capabilities`; aggregated-only data shape (REP-BR-NFR-03); machine-to-machine Authorisation via IdP principals.

Journey 5 — Edge case: cross-region fee-paid booking during partial rollout (Risk #1)

Persona: Sam (from Journey 1) needs to book an off-circuit fee-paid judge. The judge's home region (Region B) has migrated to NJI; Sam's region (Region A) is still on APEX. This scenario is unique to the rollout window — pre-rollout it doesn't apply (everyone on APEX), post-rollout it doesn't apply (everyone on NJI).

Opening scene. Sam needs to allocate a Region B fee-paid judge to a Region A vacancy.

Rising action.

1. APEX (Region A) has Sam's vacancy. NJI (Region B) has the judge's profile, ticket, and availability.
2. Per the per-wave handling decision (Risk #1 mitigation in 1600 brainstorming), this cross-boundary workflow falls back to manual coordination during the rollout window: Sam phones Region B's RSU, who confirms the judge's availability, Sam records the booking in APEX with a manual reference to the Region B judge identifier.
3. The booking processes in APEX as it always has. Region B's RSU records the booking against the judge's profile in NJI out-of-band (a known gap during the rollout).
4. When Region A migrates in a later wave, the cross-boundary workflow disappears — both sides are on the same platform.

Critical moment. The conscious decision NOT to build a transitional integration between APEX and NJI. The rollout window is bounded; the manual coordination is documented and time-limited; the simplification is preserved.

Resolution. Cross-region operations continue with documented manual handling for the rollout window only. Risk #1 is operationally managed, not architecturally solved.

Capabilities revealed: Per-wave cross-boundary handling decisions are programme-management deliverables, not application features. The system is deliberately simple about this — the edge case is paid for in operational coordination during a time-bounded window, not in transitional code.

Journey Requirements Summary

The five journeys reveal these capability areas (mapped to the 12-service decomposition):

Capability area	Services / decisions involved
Authentication via SSO + Authorisation gating per role + Region/Area	Authorisation (cross-cutting); D9 (users + roles migrated)
Absence approval workflow → automatic Vacancy creation	Absence (domain), Vacancy (domain); R4
Booking with `Vacancy.markFilled` orchestration	Booking (domain), Vacancy (domain); R5
Sitting / Booking confirmation by Court users	Sitting (domain), Booking (domain)
Payment schedule generation in JFEPS-compatible Excel	Payment (domain) with versioned content-type
Booking acknowledgement and absence acknowledgement emails	Notification (cross-cutting)
Modern UI with accessibility, responsiveness, performance	UX-override per D4
Itinerary view scoped to own profile (Judges)	Itinerary (read model); Strategy A federation
Aggregated, case-level-stripped MI Feed API for DA&I	MI Feed (read model); REP-BR-NFR-03
API-as-Product standards (versioning, `/capabilities`, deprecation policy)	All services per Phase 0 (D1)
Per-wave cross-boundary manual coordination	Programme management (not application capability); Risk #1

Domain-Specific Requirements

Compliance & Regulatory (UK govtech)

- **Accessibility — WCAG 2.2 Level AA**, required by the Public Sector Bodies (Websites and Mobile Applications) Accessibility Regulations 2018. Every domain phase delivers UI per D4; each phase's UI must be tested for WCAG 2.2 AA before cutover. APEX-era baseline is preserved at minimum; modern UI on new technology (per the user-experience uplift in the vision) targets a measurable improvement.
- **GDS Service Standard alignment** — HMCTS internal systems reference the GDS Service Standard as the bar for digital service quality. Full GDS service assessments are not always required for internal-only systems, but the principles (user research, accessibility, performance, security, simple-as-possible) apply.
- **UK GDPR and Data Protection Act 2018** — personal data scope is limited to user/judge identity, contact details, payroll numbers, and operational metadata. **JI does not hold case-level data** (REP-BR-NFR-03 from `functional-modules.md`); this remains a binding constraint.
- **HMCTS / MoJ Government Functional Standard 7 — Security** — protective marking, access control, secure development practices. Implementation aligns with HMCTS-approved technology stack and security frameworks.
- **MoJ authentication policy** — under SSO (per locked Authorisation decision), authentication policy is owned by the HMCTS IdP, not JI. JI's Admin module's password-change capability disappears (D9 +

the noted absorption of the Admin module under SSO).
- **Freedom of Information Act 2000** — JI's aggregate sitting / utilisation data is FOI-exposable; the MI Feed API is aggregate-only by contract (REP-BR-NFR-03). Case-level data is forbidden by contract; this protects against FOI scope creep into individual hearings.
- **Government / HMCTS retention schedules** — data retention is determined by HMCTS policy. Note: migrated transactional history stays in APEX (D3); new transactional data starts fresh on NJI. Retention obligations therefore span both systems during the rollout window.

Technical Constraints

- **Encryption in transit** — latest TLS only (per programme-level security guidance and the standing rule on latest SSL/TLS versions). HTTP-only endpoints rejected.
- **Encryption at rest** — for personal data (judge records, user/role records, working patterns, payroll numbers).
- **No bank details exposure** (PAY-NFR-05) — JI never stores bank details; the finance system retains them. This is a hard architectural constraint, carried from APEX.
- **No case-level data exposure** (REP-BR-NFR-03) — Reports and MI Feed are aggregate-only. Case-level identifiers are not part of the JI data model.
- **Audit minimum (MVP)** — log-based per D7. Structured user-action audit (who did what, when, with before/after values) is a post-MVP roadmap commitment, not an MVP capability.
- **AuthN delegated to HMCTS IdP via SSO**; **AuthZ owned by JI's Authorisation service** per architectural decision. User records and role/scope mappings migrated from APEX in Phase 0 (D9), keyed to IdP principal. **HMCTS IdP password policy, session policy, and account lifecycle are wholly external to JI** — owned by central HMCTS org; JI inherits whatever the IdP enforces and does not duplicate or constrain it.
- **Performance NFRs** carried from APEX page-level baselines (≤ 5 s dashboard, ≤ 10 s list/filter, ≤ 15 s batch/annual, ≤ 30 s reports/Forward Look) — already enumerated in Success Criteria.
- **No JI involvement in payment processing** — JI generates the JFEPS-shaped Excel and emails it to a Payment Authoriser; the authoriser forwards to Liberata out-of-system. JI is not in the payment chain itself, only the schedule-generation chain.

Technology Stack (locked)

- **API layer:** Java 25 (current LTS) with Spring Boot 4.
- **Runtime / orchestration:** Kubernetes — containerised deployment for every domain and cross-cutting service.
- **Cloud platform:** Microsoft Azure — all services deployed on the Azure platform. Region selection (UK South / UK West) and Azure-native service choices (e.g. AKS, Azure Container Registry, Azure Key Vault, Azure Application Insights, Azure database services) are implementation decisions in the architecture phase.
- **UI stack:** modern UI per D4; specific framework family is an implementation decision in the architecture phase, not locked here.
- **Implications worth carrying forward:**
  - Spring Boot 4 + Java 25 align well with REST-first synchronous coordination (locked architecture decision); native HTTP client, JSON content-type negotiation, and OpenAPI tooling are first-class.

- Spring Actuator endpoints can serve `/capabilities` and health/readiness probes — supports both the API-as-Product standards (versioning, capabilities) and Kubernetes liveness/readiness checks.
- Kubernetes orchestration on Azure enables the per-region phased rollout (D8) — region-scoped deployments, rolling updates, isolated rollbacks per wave.
- Azure UK regions support UK GDPR and HMCTS data-sovereignty requirements (data residency in-country); avoids the need for Standard Contractual Clauses or transfer impact assessments that would apply if data left the UK.
- Azure-native logging (Application Insights / Log Analytics) is a natural fit for the log-based audit / observability minimum (D7); structured logging conventions defined in Phase 0 should target Azure-native ingestion.

Integration Requirements

Integration	Direction	Phase	Mechanism	Notes
HMCTS IdP (SSO)	Inbound (AuthN)	Phase 0	OIDC / SAML (per HMCTS standard)	Hard dependency; must be live in Phase 0 for any user-facing demo. Risk #6 in 1600 brainstorming.
JFEPS / Liberata	Outbound (payment)	Phase 6	JFEPS-compatible Excel via HMCTS email, forwarded by Payment Authoriser	**Unchanged from APEX** — same format, same mechanism, same human-in-the-loop.
HMCTS Email infrastructure	Outbound (notifications)	Phase 0 / used Phase 1+	SMTP via HMCTS email	Booking ack, absence ack, payment schedule. Required dependency.
DA&I (MI Feed)	Outbound (data)	Phase 8	MI Feed REST API (Strategy A pull-based federation)	Replaces export-by-email; aggregate-only contract per REP-BR-NFR-03.
eLinks / HR systems	Inbound (judge data)	— (out of scope for MVP)	Manual entry (carried from APEX)	Aspirational eLinks integration (NFR-3 in source docs) is **not** in MVP scope; manual data entry continues.
Future programmes (Tribunals, Actuals, Scheduling & Listing)	Outbound (APIs)	Post-MVP	REST APIs from JI	Vision-level; specific contract design happens when programme demand crystallises.

Risk Mitigations (domain-specific)

- **Accessibility regression vs APEX.** APEX meets HMCTS accessibility commitments (HOME-NFR-04, MJ-NFR-05, etc.). NJI must match or exceed. **Mitigation:** WCAG 2.2 AA testing per UI page in each domain phase; assistive-technology compatibility (keyboard navigation, ARIA labels for tabbed content) included in acceptance tests.
- **Data-protection regression.** APEX's constraints (no bank details, no case-level data) are binding for NJI. **Mitigation:** these constraints are encoded as architectural rules — Payment service contract excludes bank fields; MI Feed and Reports schemas exclude case identifiers.
- **FOI exposure broadening.** A new API surface (MI Feed) creates new FOI questions about what data is published. **Mitigation:** MI Feed contract is aggregate-only and version-controlled; FOI scope is pre-determined by the contract, not by ad-hoc query capability.
- **Security clearance / vetting of implementation team.** Programme-management territory; not specified in this PRD. **Mitigation:** team members handling judicial / personal data work under HMCTS standard clearance levels.
- **HMCTS IdP integration timing.** SSO must be live in Phase 0; if HMCTS IdP integration slips, the MVP rollout is blocked. **Mitigation:** mock-IdP fallback for internal demo during Phase 0, contingency to wire to a different HMCTS-approved IdP if needed (carried from Risk #6 in 1600 brainstorming).
- **Reference Data + Users/Roles migration correctness** (D3 + D9 + Risk #13 + Risk #14) — already in the Risk register; restated here as a domain-specific concern because incorrect role assignments are a governance and access-control issue, not just a technical bug.

API Backend Specific Requirements

Project-Type Overview

JI is decomposed into 12 services across three clusters:

- **Domain services** (write surfaces): Judge, Absence, Vacancy, Booking, Sitting, Payment.
- **Cross-cutting services**: Reference Data, Authorisation, Configuration, Notification.
- **Read-model services** (federated): Itinerary, MI Feed.

Every service is API-first, exposes a versioned contract, and is callable by both the modern UI (per D4) and external consumers (DA&I, future programmes per the strategic-platform vision). The 12-service decomposition is the durable product surface.

Technical Architecture Considerations

- **Coordination style: REST-first synchronous.** Domain services call each other directly when they need to coordinate (e.g. Booking → `Vacancy.markFilled`). No domain event stream, no message bus, no webhook fabric. This is a deliberate locked decision from the brainstorming session.
- **Read-model federation strategy: Strategy A — fan-out at request time.** Itinerary and MI Feed hold no data of their own; every read fans out to the domain services in parallel and composes the answer. Strategy C (cached projection) is a designed fallback if Forward Look misses the ≤ 30 s NFR (Risk #9).
- **Service-to-service trust:** internal calls within the cluster authenticate via service-token / mTLS (specific mechanism is an architecture-phase decision); external calls authenticate via the same SSO/IdP-derived principal as user calls.
- **Idempotency:** write operations that may be retried (e.g. `POST /bookings`, `POST /payments/process`) accept an idempotency key. Specific mechanism is an architecture-phase

decision.

Endpoint Specifications

Endpoint shape is illustrative — definitive contracts are produced as Phase 0 paper artefacts (per D1) and in each domain phase as the service is built.

Cross-cutting services (Phase 0):

Service	Representative endpoints
Reference Data	`GET /reference-data/regions`, `/offices`, `/judicial-vocabularies`, `/calendar`; admin-gated `POST/PUT` writes
Authorisation	`POST /authz/check`, `GET /users/{id}/effective-permissions`
Configuration	`GET /config/{key}` (read-mostly typed policy values)
Notification	`POST /notifications/send` (transactional emails: booking ack, absence ack, payment schedule)

Domain services (Phases 1–6):

Service	Representative endpoints
Judge	`POST/GET/PUT /judges`, `POST /judges/{id}/working-patterns`, `POST /judges/{id}/tickets`
Absence	`POST/GET /absences`, `POST /absences/{id}/approve`, `POST /absences/{id}/extend` *(sickness only)*
Vacancy	`POST/GET /vacancies`, `POST /vacancies/{id}/markFilled` *(called by Booking, R5)*, `POST /vacancies/{id}/cancel`
Booking	`POST /bookings` *(accepts optional `vacancyId` and orchestrates `Vacancy.markFilled`)*, `POST /bookings/{id}/confirm`, `POST /bookings/{id}/cancel`
Sitting	`POST/GET /sittings`, `POST /sittings/{id}/confirm`, `POST /sittings/{id}/verify`, `POST /sittings/{id}/split` *(AM/PM)*
Payment	`POST /payments/process` *(eligible bookings → schedule)*, `GET /payments/{id}/schedule` *(content-type negotiated)*, `POST /payments/{id}/reconcile`

Read-model services (Phases 7–8):

Service	Representative endpoints
Itinerary	`GET /itineraries/courts/{officeId}` *(monthly / annual)*, `GET /itineraries/judges/{judgeId}`, `GET /itineraries/forward-look`

Service	Representative endpoints
MI Feed	`GET /reporting/sittings`, `GET /reporting/utilisation`, `GET /reporting/vacancies`, `GET /reporting/bookings` *(all aggregate-only per REP-BR-NFR-03)*

Authentication Model

- **End-user authentication: HMCTS IdP via SSO** (OIDC or SAML — exact mechanism is HMCTS-IdP-side; JI integrates with whichever the IdP exposes).
- **End-user authorisation: JI's Authorisation service.** Migrated APEX users (per D9) are keyed to the IdP principal. Every domain call resolves the principal's roles + Region/Area scope via Authorisation before authorising the action.
- **HMCTS IdP password / session / account lifecycle policies are wholly external to JI** (per Step 5 Technical Constraints) — JI inherits whatever the IdP enforces.
- **Service-to-service authentication:** TBD in architecture phase; mTLS or service-token is the typical fit for the chosen Java/Spring/Kubernetes stack.
- **External consumer authentication** (DA&I MI Feed, future programmes): same IdP principal model where possible; service principals or API keys are an architecture-phase fallback if the IdP doesn't issue principals for non-human consumers.

Data Schemas

Canonical representation: **JSON** for all REST endpoints. Specific resource schemas (Judge, Absence, Vacancy, Booking, Sitting, Payment, Itinerary, Reporting feed) are produced as Phase 0 paper contracts (per D1) and refined per phase.

Versioned content-types are first-class for shape-sensitive resources:

- `GET /payments/{id}/schedule` accepts `application/vnd.hmcts.jfeps+json` (canonical JI shape) or `application/vnd.hmcts.jfeps+xlsx` (format-shifted JFEPS Excel for Liberata workflow). The JFEPS shape evolves independently of Payment internals.
- Other resources may grow versioned content-types over time as integration partners require shape-stability.

Forbidden fields by contract:

- Bank details (PAY-NFR-05) — not in any Payment resource shape.
- Case-level identifiers (REP-BR-NFR-03) — not in Reports or MI Feed shapes.

Error Codes

- **HTTP status codes** semantically — 200/201 for success, 400 for validation errors, 401/403 for auth, 404 for not-found, 409 for conflict (e.g. double-booking attempts blocked by FPB-NFR-04), 422 for semantically valid but business-rule-rejected, 5xx for server-side faults.
- **RFC 7807 Problem Details for HTTP APIs** as the standard error envelope — `type`, `title`, `status`, `detail`, `instance`, plus problem-specific extension fields where useful.
- Architecture-phase decision: define the specific problem `type` URIs for the cross-cutting categories (validation failure, authorisation failure, business-rule rejection, dependency failure, etc.).

Rate Limits

TBD — architecture-phase decision. For internal HMCTS systems with a bounded user population, rate limits are typically low priority for human traffic; the relevant cases (per-service-principal limits for MI Feed, burst protection against runaway scripts) will be specified in the architecture phase.

API Versioning

- **Versioning policy is a Phase 0 deliverable** as part of API-as-Product standards (per D1).
- Working assumption (architecture-phase confirmable): versioning via the URI path prefix (e.g. `/v1/judges`, `/v2/judges`) for major versions; backwards-compatible additions within a major version don't require a new path.
- **Deprecation policy** is part of the same Phase 0 artefact: deprecated endpoints emit a `Deprecation` header, are documented, and are retired no sooner than N months after first deprecation notice (specific N TBD).
- **`/capabilities` endpoint** (per locked architecture decision) — every service exposes its current contract version, deprecated capabilities, and the deprecation timeline.

Client Tooling

- **API testing during build: Postman.** Postman collections are the primary client for validating APIs end-to-end as each service is built, ahead of the UI layer being wired in. Each phase produces a Postman collection that exercises the phase's endpoints against running services in the deployment platform.
- **UI layer client:** TBD in architecture phase. UI stack itself is not locked (per Step 5 Technology Stack — UI framework is an architecture-phase decision); specific UI-client tooling (e.g. generated TypeScript client from OpenAPI) follows from that decision.
- **External consumer clients** (DA&I, future programmes): direct REST calls in their native stack are sufficient; no formal SDK is required for MVP.

API Documentation

- **OpenAPI 3.x specifications** generated from each service's code (Spring Boot has first-class OpenAPI tooling).
- **Each service's `/capabilities` endpoint** exposes the current contract version and lifecycle metadata.
- **Documentation hosting:** Swagger UI per service for developer onboarding, plus a published consolidated API catalog (architecture-phase decision on hosting / branding).
- **Postman collections** per phase serve double duty as test artefacts and as practical, executable API documentation for stakeholders ahead of UI demos.

Implementation Considerations

- **Stack alignment** (per Step 5 Technology Stack): Java 25 + Spring Boot 4 + Kubernetes on Azure provides first-class support for every requirement above — Spring Web for REST endpoints, Spring Security for AuthZ integration, Spring Actuator for `/capabilities` and Kubernetes probes, springdoc-openapi for OpenAPI generation, Azure API Management for cross-cutting concerns (rate limits, header injection, deprecation header policy) if needed.

- **Per-service deployment unit.** Each of the 12 services is a containerised Spring Boot app on Kubernetes; per-region rollout (D8) is enabled by region-scoped namespaces / clusters or by service-instance-level region targeting (architecture-phase choice).
- **Phase 0 platform smoke-test** (per the executive summary) — Reference Data exercises every API-as-Product standard (versioning, content-type negotiation, `/capabilities`, RFC 7807 errors) before any domain service is built. Phase 0 is not just foundations; it's the standards-validation phase.

Project Scoping & Phased Development

This section extends the **Product Scope** section above (MVP / Growth / Vision split) with strategic context: MVP philosophy, phase-by-phase journey mapping, and risk-based scoping. The scope split itself is unchanged.

MVP Strategy & Philosophy

MVP approach: NJI (New JI MVP). The MVP is "complete enough that an entire region — every applicable role, every operational workflow — can move off APEX and stay off it."

This is constrained by the project's nature, not chosen from a menu of MVP archetypes. For a brownfield rebuild replacing an existing operational system with a fixed end-of-life, the MVP cannot be smaller than "fully functional for one region's worth of users" — anything less means that region cannot migrate, which means APEX cannot retire, which means the programme has not begun delivering its primary value (D2 + D8).

Resource requirements: TBD. Programme-management territory; not specified in this PRD. The two viable structures from the brainstorming session are:

- **Variant α** (single squad, sequential build): Phase 0 → Judge → Absence → Vacancy → Booking → Sitting → Payment → Itinerary → MI Feed → wave 1 rollout, in strict sequence. Lower coordination overhead; longer calendar.
- **Variant β** (multi-squad, Sitting parallel from Phase 2): same dependency order, but Sitting is co-developed with Booking. Compresses calendar; requires 2+ squads.

The default is α with β as a capacity-conditional upgrade once staffing is known.

Phase-by-Phase Journey Mapping

Mapping the **5 user journeys** (Step 4) to the **build phases** (from the brainstorming session migration table). A journey becomes *demoable* at the end of the phase that completes its last dependency.

Journey	Demoable at end of	Dependency
Journey 3 — Judge views itinerary	Phase 7 (Itinerary)	Federates over Judge + Absence + Vacancy + Booking + Sitting
Journey 2 — Court daily sitting confirmation	Phase 5 (Sitting)	Sitting + Authorisation; Phase 5 ends with confirmation flow live

Journey	Demoable at end of	Dependency
Journey 1 — RSU cover-creation through payment (canonical operational cycle)	Phase 6 (Payment)	Full operational chain Judge → Absence → Vacancy → Booking → Sitting → Payment must be live
Journey 4 — DA&I MI Feed API consumer	Phase 8 (MI Feed)	MI Feed federates over all domain services including Payment
Journey 5 — Cross-region edge case during partial rollout	Phase 9+ (rollout window only)	Only relevant once at least one region has migrated; resolves once last region migrates

Implication for stakeholder communication: the canonical operational demo (Journey 1) is not available until Phase 6. Phases 1–5 produce per-module demos against partial chains (e.g. Phase 1 demos Judge management with working-pattern-generated sittings; Phase 4 demos vacancy → booking but cannot yet demo confirmation → payment). This is a programme-management consideration, not a scope decision.

Risk-Based Scoping

The 1600 brainstorming session's risk register applies in full; these are the scoping-level risks specifically.

Technical risks (and mitigation via scope):

- **Strategy A read-model federation may miss the ≤ 30 s Forward Look NFR** (Risk #9). Mitigation already in scope: Strategy C cache fallback is a *designed* fallback; switched on if Phase 7 measurement shows the risk materialising. No scope change required up-front.
- **Reference Data + Users/Roles migration correctness** (Risk #13). Mitigation in scope: Phase 0 includes named-owner sign-off as a deliverable, not an afterthought.
- **APEX ⇄ IdP identity mapping** (Risk #14). Mitigation in scope: Phase 0 reconciliation report with explicit handling rules for unmatched records.

Programme / market risks (and mitigation via scope):

- **Strategic-platform vision** (one external HMCTS programme integrating via API by post-MVP) is *aspirational at MVP*. The MVP itself ships without external API consumers actually integrated; the API surface is in place, but consumer onboarding is post-MVP. This is consistent with Step 3 Growth Features.
- **Cross-region workflow during partial rollout** (Risk #1). Mitigation in scope (and out of application scope): per-wave manual coordination is a programme-management deliverable, not an application feature. Application stays simple; coordination is paid for in operations during the time-bounded rollout window.

Resource risks (and mitigation via scope):

- **Variant β is a capacity-conditional upgrade, not a default.** The plan does not assume multi-squad capacity; if capacity is constrained, the default α plan still delivers the same MVP content on a longer calendar.

- **HMCTS IdP integration timing** (Risk #6). Mitigation in scope: mock-IdP fallback for internal demo during Phase 0, contingency to wire to a different HMCTS-approved IdP if needed.

Scope Confirmation

No requirement from the brainstorming session, the as-is docs, or the user's locked decisions (D1–D9) has been silently de-scoped or moved to a later phase by this PRD. The MVP / Growth / Vision split is exactly as established in Step 3 Product Scope, and the build / rollout sequence is exactly as established in the brainstorming migration table. This step adds strategic framing (MVP philosophy, journey mapping, risk-based scoping) without altering scope.

Functional Requirements

This section is the binding capability contract for NJI. UX, architecture, and epic breakdown will all trace back to these requirements. A capability not listed here will not exist in the final product unless explicitly added.

Identity & Authorisation

- **FR1**: Authenticated users access NJI via HMCTS IdP single sign-on; password, session, and account lifecycle are owned by the IdP and not duplicated in NJI.
- **FR2**: NJI's Authorisation service maps each authenticated principal to one or more roles and a Region/Area scope, and authorises every system call against that mapping.
- **FR3**: Authorised users can retrieve their effective permissions for their authenticated session.
- **FR4**: System administrators can update role and Region/Area assignments for migrated and new users.
- **FR5**: External (machine-to-machine) consumers can authenticate via service principals issued by the HMCTS IdP, with their authorisation scoped through the same Authorisation service.

Foundational Data Management

- **FR6**: RSU users can view and maintain Reference Data lists — Regions, Offices, judicial vocabularies, calendar / financial-year boundaries — with named-owner sign-off on changes.
- **FR7**: Every NJI service can read Reference Data via a versioned API; Reference Data is the single writer (no duplicates anywhere in the system).
- **FR8**: Authorised administrators can read and update typed Configuration policy values (e.g. session timeout warnings, batch schedules, feature flags).
- **FR9**: NJI dispatches transactional emails (booking acknowledgements, absence acknowledgements, payment schedules) via HMCTS email infrastructure, with a delivery log retained.

Judge Records & Working Patterns

- **FR10**: RSU users can search and filter judges by name, base location, location type, and judge type.
- **FR11**: RSU users can maintain judge profiles, including personal details, judge type, base office, active/inactive status, and role-specific data (payroll number, retirement date, fee-payment status, London weighting, name-for-itinerary, heading).
- **FR12**: Authorised users can define and update Working Patterns (None / Daily / Weekly) with target sit %, jurisdictional split, and per-day work-type pattern.

- **FR13**: NJI auto-populates judge itineraries up to the next 31st March from the working pattern, preserving any prior absences.
- **FR14**: RSU users can convert salaried judges between full-time and part-time, adjusting mandatory sitting days.
- **FR15**: RSU users can maintain ticket information per judge role, requiring start date and ticket type.
- **FR16**: NJI validates that jurisdictional split percentages total 100% before saving.
- **FR17**: RSU users can switch a judge's base location to another office within the same Region; cross-Region changes require OPT Advice Point and are out-of-system.
- **FR18**: Authorised users can link to judges managed by other offices (off-circuit / cross-Region) for booking purposes.

Absence Workflow

- **FR19**: Authorised users (RSU, Court, Judges where permitted) can record absence requests with start/end date, partial-day option (full / AM / PM), type from a controlled list, and an NTBF flag.
- **FR20**: NJI distinguishes auto-confirmed absences (from judicial teams) from those requiring confirmation (from Courts or judges); confirmation can trigger an acknowledgement email.
- **FR21**: Sickness absences can be extended without creating a new record; non-sickness extensions require a new absence record.
- **FR22**: Authorised users can mark absences as *Not To Be Filled* (NTBF) or as *needs fee-paid cover*.

Vacancy & Cover

- **FR23**: NJI auto-creates a vacancy when an approved absence requires fee-paid cover, pre-populated with judge type, work type, ticket, and dates.
- **FR24**: Authorised users can create standalone vacancies independent of any absence.
- **FR25**: Authorised users can edit a vacancy's daily breakdown — cancel individual days with a captured reason; extend or shorten the period.
- **FR26**: NJI marks a vacancy as filled when a booking is created against it; vacancy days cannot be cancelled once a booking is recorded.
- **FR27**: NJI surfaces fee-paid judges matching a vacancy's filter as a hint for advertising; advertising itself is performed out-of-system by judicial teams.
- **FR28**: Authorised users can cancel or close vacancies (e.g. when a parent absence becomes NTBF).

Booking Management

- **FR29**: Authorised users can create fee-paid bookings (linked to a vacancy or standalone), capturing judge, court, date, session type (full / AM / PM / evening / reserved-matter), booking type, and work type.
- **FR30**: Booking creation orchestrates `Vacancy.markFilled` synchronously when a `vacancyId` is supplied.
- **FR31**: NJI tracks booking status (planned, provisional, confirmed, cancelled, rejected) with reason capture for cancellation.
- **FR32**: NJI sends booking acknowledgement emails to fee-paid judges, batched overnight or sent immediately via *Create and Email Now*.
- **FR33**: NJI requires a Y/N fee-payment answer at booking time when a judge's fee-payment status is *Ask when booking*.
- **FR34**: NJI prevents double-booking of fee-paid judges for overlapping sessions.

Sitting Management

- **FR35**: NJI generates planned sittings for salaried judges from their working patterns, court, date, and work type.
- **FR36**: Authorised users can filter sitting records by Region/Office, judge type, judge, and date range.
- **FR37**: Authorised users can confirm that a sitting actually took place, updating outcome (confirmed, cancelled, rejected) and actual work type.
- **FR38**: Authorised users can split a sitting into AM/PM with different work types within a single day.
- **FR39**: Authorised users can create ad-hoc sittings for salaried judges, including DJ(MC)s and Legal Advisers in County Courts.
- **FR40**: Verifiers can verify confirmed sittings; once verified, the data is read-only and amendments require an RFC.

Payment & Reconciliation

- **FR41**: Authorised users can list confirmed bookings and salaried sittings eligible for payment, filterable by Region/Office, judge, date range, and payment status (pending, requested, paid).
- **FR42**: Authorised users can mark eligible bookings as *payment requested* individually or in bulk.
- **FR43**: NJI generates JFEPS-compatible payment schedules and dispatches them as Excel attachments to a chosen Payment Authoriser via email; the Payment Authoriser forwards to Liberata out-of-system.
- **FR44**: NJI exposes the payment schedule via API with content-type negotiation (`application/vnd.hmcts.jfeps+json` or `+xlsx`); the JFEPS shape evolves independently of Payment internals.
- **FR45**: NJI prevents double submission of the same booking for payment.
- **FR46**: Authorised users (Finance, RSU) can flag payments as reconciled, capturing notes for mismatches; once fully reconciled, a payment cannot be re-requested for the same booking.
- **FR47**: NJI does not store or expose bank details for any judge — those remain in the finance system.

Itineraries & Reporting (Read Models)

- **FR48**: Authorised users can render the Court Itinerary (monthly or annual) for a given Office, Financial Year, and Month, showing sittings, bookings, vacancies, and NTBF absences for each day.
- **FR49**: Authorised users can render the Judge Itinerary for one or more judges over a date range, scoped by Authorisation (judges see only their own; courts see their office; RSU sees their region).
- **FR50**: Authorised users can use the Forward Look view across a Region with paged or filtered access for performance.
- **FR51**: Itinerary cells are clickable and drill into the underlying record (Sitting, Absence, Vacancy, or Booking).
- **FR52**: Authorised users can copy/export Itinerary and Report contents to Excel and PDF.
- **FR53**: NJI provides a fixed catalogue of standard Reports (weekly sitting projections, weekly vacancies, absence analysis, vacancy by court, confirmed sittings/bookings by judge or judge type, judge utilisation, jurisdictional split, summary by court / work type) with parameter filters per report.
- **FR54**: NJI exposes aggregated MI Feed APIs for external consumers (DA&I, future programmes); MI Feed responses contain no case-level data and are aggregate-only by contract.

Platform Operations & Migration

- **FR55**: Authenticated users land on a Home page showing role-scoped navigation, Region/Area selector, summary tiles for the selected scope (judges, absences, vacancies, pending payments, payments made, unreconciled), and contextual help.
- **FR56**: NJI's UI replicates the functional surface of the as-is APEX UI on a modern UI stack and meets WCAG 2.2 Level AA accessibility standards.
- **FR57**: NJI migrates Reference Data and active user records — including role and Region/Area mappings keyed to HMCTS IdP principals — from APEX as a Phase 0 deliverable, with named-owner sign-off; unmatched user records are flagged for explicit handling (drop / hold / manual map).
- **FR58**: NJI supports per-region phased activation — a region's user accounts can be activated for NJI use only when that region's feature-parity gate is passed; activation is a flag flip, not a data migration.
- **FR59**: Every NJI service exposes a versioned API contract, a `/capabilities` endpoint, RFC 7807 problem-details for errors, and a published OpenAPI specification.
- **FR60**: Every NJI service emits structured logs with correlation IDs and consistent error categorisation, retained for pilot incident triage.
- **FR61**: Every NJI domain service has acceptance tests that verify behavioural parity with APEX, executed with real APEX running as the oracle.

Non-Functional Requirements

Performance

Page-level NFRs are carried from the APEX baseline (`functional-modules.md` cross-cutting NFRs); NJI must match or exceed each.

- **NFR1 — Static page load:** ≤ 3 s for static UI loads (e.g. Home initial render).
- **NFR2 — Dashboard refresh:** ≤ 5 s when Region/Area selection changes.
- **NFR3 — List / filter operations:** ≤ 10 s for typical operational lists (judges, absences, vacancies, bookings, sittings, payments) at Region scope.
- **NFR4 — Batch / annual operations:** ≤ 15 s (e.g. annual itinerary render, batch payment-request processing).
- **NFR5 — Reports / Forward Look:** ≤ 30 s for standard report parameters and for the Forward Look view at Region scope.
- **NFR6 — Single-resource API read:** ≤ 500 ms p95 (e.g. `GET /judges/{id}`).
- **NFR7 — Domain write API:** ≤ 1 s p95 for typical write operations (excluding orchestrated cross-service calls like Booking → Vacancy.markFilled).
- **NFR8 — Federated read (Itinerary, Forward Look):** ≤ 30 s p95 under Strategy A. Strategy C cache fallback is pre-designed and switched on if measurement shows the p95 breached (Risk #9).
- **NFR9 — Capacity (rough order-of-magnitude):** concurrent users per region ~50–100; national concurrent users ~200–500 once all regions migrated. Burst capacity for monthly verification deadlines accounted for.

Security

- **NFR10 — Transport encryption:** Latest TLS only on every endpoint; HTTP-only endpoints rejected.
- **NFR11 — Data-at-rest encryption:** All personal data (judge records, user/role records, working patterns, payroll numbers, payment metadata) encrypted at rest.

- **NFR12 — Authentication:** All human users authenticated via HMCTS IdP SSO (per FR1). Service-to-service authentication (mTLS or service token) enforced for internal calls; mechanism is an architecture-phase choice.
- **NFR13 — Authorisation enforcement:** Every API call resolves the principal's roles + Region/Area scope through the Authorisation service; no operation bypasses this check.
- **NFR14 — Forbidden data scope:** No bank details stored or exposed by any service (PAY-NFR-05). No case-level data in any read model or report (REP-BR-NFR-03).
- **NFR15 — Government Functional Standard 7 alignment:** NJI aligns with HMCTS / MoJ Government Functional Standard 7 — Security, including protective marking, access control, and secure development practices.
- **NFR16 — Secret management:** Service credentials, signing keys, and integration secrets stored in a managed secret store (Azure Key Vault or equivalent); never in source control or environment-baked images.

Accessibility

- **NFR17 — WCAG 2.2 Level AA:** Every UI page meets WCAG 2.2 Level AA accessibility standards; tested per UI page in each domain phase before that phase's gate is passed.
- **NFR18 — Assistive technology compatibility:** Keyboard navigation, ARIA labels for tabbed and dynamic content, and screen-reader compatibility per HMCTS accessibility standards.
- **NFR19 — Public Sector Bodies Accessibility Regulations 2018:** NJI complies with the Public Sector Bodies (Websites and Mobile Applications) (No. 2) Accessibility Regulations 2018, including publication of an accessibility statement.

Integration

- **NFR20 — HMCTS IdP integration:** Hard Phase 0 dependency. NJI integrates with whichever AuthN protocol the HMCTS IdP exposes (OIDC or SAML).
- **NFR21 — JFEPS / Liberata integration unchanged:** Payment schedule format (JFEPS-compatible Excel), email-to-Authoriser delivery, and authoriser-forwards-to-Liberata workflow are preserved exactly as in APEX. No format change for finance.
- **NFR22 — HMCTS email infrastructure:** Outbound transactional emails (booking ack, absence ack, payment schedules) dispatch via HMCTS email; delivery is reliable but not low-latency-critical (overnight batch acceptable for booking acknowledgements).
- **NFR23 — DA&I MI Feed:** Aggregate-only REST API contract; no case-level data exposed under any consumer authorisation.
- **NFR24 — eLinks / HR systems:** No automated integration in MVP scope; manual data entry by RSU continues, matching the APEX-era pattern.

Observability (MVP minimum per D7)

- **NFR25 — Structured logging:** Every service emits structured logs with consistent fields, correlation IDs threaded through service-to-service calls, and a defined error-categorisation taxonomy. Logging schema is a Phase 0 deliverable.
- **NFR26 — Log retention:** Logs retained sufficient for pilot incident triage; specific retention period set in Phase 0 within HMCTS data-retention policy.
- **NFR27 — Log ingestion:** Logs ingested into Azure-native logging (Application Insights / Log Analytics).

- **NFR28 — Health and readiness probes:** Every service exposes Kubernetes-compatible liveness and readiness endpoints (Spring Actuator).
- **NFR29 — Roadmap commitments (post-MVP, not in MVP):** Structured user-action auditing (who-did-what-when with before/after values for write operations) is a post-MVP roadmap commitment per D7. Metrics and trace observability beyond logs is post-MVP.

Data Privacy & Sovereignty

- **NFR30 — UK GDPR / Data Protection Act 2018 compliance:** Personal data scope is limited to user/judge identity, contact details, payroll numbers, and operational metadata. No case-level data anywhere in NJI.
- **NFR31 — Data residency:** All NJI services and data hosted in Azure UK regions only (UK South / UK West). No personal data leaves the UK.
- **NFR32 — Retention:** Data retention per HMCTS retention schedules. Migrated transactional history remains in APEX (D3); NJI retains only data created in NJI from migration onward.
- **NFR33 — FOI scope:** Aggregate operational data exposable per FOI requests; case-level data is forbidden by contract (REP-BR-NFR-03) and therefore outside FOI scope by construction.

Reliability & Availability

- **NFR34 — Operational availability:** NJI is available during HMCTS operational hours (typically 07:00–19:00 UK weekdays). Out-of-hours availability is best-effort, not contracted.
- **NFR35 — Payment-cycle continuity:** Zero failed JFEPS payment cycles attributable to NJI deployment, rollout, or runtime issues. Payment generation can fall back to manual handling within a payment cycle if NJI is unavailable, but this is an operational contingency, not a normal-mode expectation.
- **NFR36 — Per-wave rollback:** Each rollout wave (Phase 9, 10, …) has a documented rollback path returning the affected region to APEX within one operational cycle if the wave's gate is breached post-cutover.
- **NFR37 — Strategy A degraded-mode contract:** If federated read latency breaches NFR8, NJI degrades to Strategy C cached projection rather than failing; cache freshness window is published per `/capabilities`.
- **NFR38 — Region-isolated deployments:** A failure or rolling deployment in one region's NJI instance does not affect other regions.

Maintainability

- **NFR39 — API-as-Product standards:** Every service exposes versioned contracts, `/capabilities`, RFC 7807 error envelopes, and a published OpenAPI specification (per FR59). Versioning and deprecation policy is a Phase 0 deliverable.
- **NFR40 — Per-service deployment unit:** Each of the 12 services is independently deployable on Kubernetes; rolling updates per service per region without coupling.
- **NFR41 — Behavioural-parity test suite:** Every domain service has acceptance tests using real APEX as oracle (per FR61); the suite runs as a gate before each rollout wave's cutover.
- **NFR42 — Postman collections:** Each phase produces a Postman collection that exercises the phase's endpoints; collections are versioned alongside the services.

Decisions Log (D1–D9)

These are the 9 locked decisions taken during the 2026-05-05 brainstorming follow-up. Each is referenced inline throughout the PRD; the consolidated list is here for navigability.

ID	Decision	Implication
D1	Phase 0 Foundations scope locked: Reference Data, Authorisation (with SSO), Configuration, Notification, API contracts, deployment platform, structured logging conventions. Audit & metrics/trace observability post-MVP.	Sets what must be in place before any domain service is built.
D2	Cutover strategy: phased rollout. Migrated users do not use APEX; non-migrated users do not use NJI.	No dual-write coexistence; risk amortised across waves.
D3	Data migration: Reference Data only (extended by D9). No transactional data migration.	Each region migrates onto a clean transactional state; historical data stays in APEX.
D4	Feature-parity gate is functional + UI-replicates-APEX (modern UI stack, no redesign).	UX/visual_design/user_journeys are in scope (override on `api_backend` classification).
D5	APEX is the behavioural reference for acceptance testing, not a migration host.	Acceptance tests run with real APEX as oracle; behavioural parity is the gate.
D6	APEX maintenance is out of project scope.	APEX is a stable external system in the project plan; not co-managed.
D7	Audit / Observability MVP minimum: log-based (request, error). User-action audit on the post-MVP roadmap.	Structured logging is a Phase 0 deliverable; metrics/traces deferred.
D8	Rollout boundary: by region, all applicable user roles.	A region migrates only when every in-region role's functionality is complete.
D9	Users + roles migrated from APEX in Phase 0 to seed Authorisation, keyed to HMCTS IdP principal.	Authorisation is testable end-to-end with realistic data from day 1; per-wave activation is a flag flip.

Glossary

Term	Meaning
APEX	Oracle Application Express; the legacy platform JI runs on today
DA&I	Data, Analysis & Insight; HMCTS analytics / MI team consuming JI data
DJ	District Judge
DJ(MC)	District Judge (Magistrates' Courts)

Term	Meaning
FOI	Freedom of Information Act 2000
FPB	Fee-paid and other Bookings (APEX module name)
GDS	Government Digital Service (UK Cabinet Office)
HMCTS	His Majesty's Courts and Tribunals Service
IdP	Identity Provider (HMCTS's SSO / authentication system)
JFEPS	Judicial Fee Payment System (HMCTS finance system)
JFL	Judges Forward Look (sub-module of Judge Itinerary)
JI	Judicial Itineraries (the existing APEX system)
Liberata	HMCTS's payment processing partner
MI	Management Information
MoJ	Ministry of Justice
NJI	New JI — the API-driven rebuild this PRD describes
NTBF	Not To Be Filled (an absence flag — cover not required)
OIDC	OpenID Connect (an authentication protocol)
OPT	One Performance Truth; the broader Oracle/APEX platform JI sits on
RFC	Request for Change (process for amending verified sitting data)
RFC 7807	IETF specification for Problem Details for HTTP APIs
RSU	Regional Support Unit (HMCTS regional admin teams)
S&L	Scheduling & Listing reforms (HMCTS programme)
SAML	Security Assertion Markup Language (an authentication protocol)
SSO	Single Sign-On
TBD	To Be Determined (programme-management or architecture-phase decision)
UK GDPR	UK General Data Protection Regulation (post-Brexit equivalent of EU GDPR)
WCAG	Web Content Accessibility Guidelines

References

Source documents consulted during PRD generation (also recorded in this PRD's `inputDocuments` frontmatter):

- `_bmad-output/brainstorming/brainstorming-session-2026-05-05-1600.md` — the 9 locked decisions, 12-service decomposition, migration table, and risk register
- `docs/architecture/asis/functional-modules.md` — catalogue of the 12 functional modules in the existing Oracle APEX JI system

- `docs/architecture/asis/data-dependencies.md` — JI's external data dependencies (eLinks, HR, JFEPS, Liberata, DA&I, HMCTS Email)
- `docs/architecture/asis/integration-dependencies.md` — JI's integration flows and mechanisms

The 1600 brainstorming session itself supersedes lines 139–149 of the 2026-05-01 brainstorming session (migration sequencing) and the entirety of the 2026-05-05-1500 brainstorming draft (which was based on a strangler-fig assumption that has been retracted).